



PDF Download
3768577.pdf
03 April 2026
Total Citations: 1
Total Downloads: 279

Latest updates: <https://dl.acm.org/doi/10.1145/3768577>

RESEARCH-ARTICLE

CodeS: Natural Language to Code Repository via Multi-Layer Sketch

DAOGUANG ZAN, University of Chinese Academy of Sciences, Beijing, China

AILUN YU, Peking University, Beijing, China

WEI LIU, Peking University, Beijing, China

DONG CHEN, Huawei Technologies Co., Ltd., Shenzhen, Guangdong, China

BO SHEN, Huawei Technologies Co., Ltd., Shenzhen, Guangdong, China

YAFEN YAO, Huawei Technologies Co., Ltd., Shenzhen, Guangdong, China

[View all](#)

Open Access Support provided by:

Chinese Academy of Sciences

Peking University

University of Chinese Academy of Sciences

Shandong University

Huawei Technologies Co., Ltd.

Published: 19 September 2025
Accepted: 14 September 2025
Revised: 25 August 2025
Received: 20 July 2024

[Citation in BibTeX format](#)

CODES: Natural Language to Code Repository via Multi-Layer Sketch

DAOGUANG ZAN*, Institute of Software, Chinese Academy of Sciences; University of Chinese Academy of Sciences, China

AILUN YU and WEI LIU, Peking University, China

DONG CHEN, BO SHEN, and YAFEN YAO, Huawei Technologies Co., Ltd, China

WEI LI and XIAOLIN CHEN, Institute of Software, Chinese Academy of Sciences, China

YONGSHUN GONG, Shandong University, China

BEI GUAN, Institute of Software, Chinese Academy of Sciences, China

ZHIGUANG YANG, Funcun-wuyou Technologies Co., Ltd, China

YONGJI WANG, Institute of Software, Chinese Academy of Sciences, China

LIZHEN CUI, Shandong University, China

QIANXIANG WANG, Huawei Technologies Co., Ltd, China

The impressive performance of large language models (LLMs) on code-related tasks has shown the potential of fully automated software development. In light of this, we introduce a new software engineering task, namely *Natural Language to code Repository* (NL2Repo). This task aims to generate an entire code repository from its natural language requirements. To address this task, we propose a simple yet effective framework CODES, which decomposes NL2Repo into multiple sub-tasks by a multi-layer sketch. Specifically, CODES includes three modules: RepoSketcher, FileSketcher, and SketchFiller. RepoSketcher first generates a repository's directory structure for given requirements; FileSketcher then generates a file sketch for each file in the generated structure; SketchFiller finally fills in the details for each function in the generated file sketch. To rigorously assess CODES on the NL2Repo task, we carry out evaluations through both automated benchmarking and manual feedback analysis. For benchmark-based evaluation, we craft a repository-oriented benchmark, SketchEval, and design an evaluation metric, SketchBLEU. For feedback-based evaluation, we develop a VSCode plugin for CODES and engage 30 participants in conducting empirical studies. Extensive experiments prove the effectiveness and practicality of CODES on the NL2Repo task.

CCS Concepts: • **Software and its engineering** → **Software libraries and repositories**; *Software implementation planning*; **Automatic programming**; *Open source model*; • **Computing methodologies** → *Natural language processing*.

*The first four authors contributed equally to this work. Corresponding authors: Daoguang Zan and Bei Guan.

Authors' Contact Information: Daoguang Zan, daoguang@iscas.ac.cn, Institute of Software, Chinese Academy of Sciences; University of Chinese Academy of Sciences, Beijing, Beijing, China; Ailun Yu, yuailun@pku.edu.cn; Wei Liu, weiliu@stu.pku.edu.cn, Peking University, Beijing, Beijing, China; Dong Chen, chendong108@huawei.com; Bo Shen, shenbo21@huawei.com; Yafen Yao, yaoyafen2@huawei.com, Huawei Technologies Co., Ltd, Beijing, Beijing, China; Wei Li, liwei224@mails.ucas.ac.cn; Xiaolin Chen, chenxiaolin2019@iscas.ac.cn, Institute of Software, Chinese Academy of Sciences, Beijing, Beijing, China; Yongshun Gong, ysgong@sdu.edu.cn, Shandong University, Jinan, Shandong, China; Bei Guan, guanbei@iscas.ac.cn, Institute of Software, Chinese Academy of Sciences, Beijing, Beijing, China; Zhiguang Yang, yangzhiguang_funcun@outlook.com, Funcun-wuyou Technologies Co., Ltd, Beijing, Beijing, China; Yongji Wang, ywang@itechs.iscas.ac.cn, Institute of Software, Chinese Academy of Sciences, Beijing, Beijing, China; Lizhen Cui, clz@sdu.edu.cn, Shandong University, Jinan, Shandong, China; Qianxiang Wang, wangqianxiang@huawei.com, Huawei Technologies Co., Ltd, Beijing, Beijing, China.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, or post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2025 Copyright held by the owner/author(s).

ACM 1557-7392/2025/9-ART

<https://doi.org/10.1145/3768577>

Additional Key Words and Phrases: Natural Language to Code Repository, Multi-Layer Sketch, Code Large Language Model, Instruction Tuning

1 INTRODUCTION

Large language models (LLMs) have achieved remarkable advancements in generating code based on natural language (NL2Code) [3, 4, 6, 14, 22, 23, 27, 31, 37, 48]. For instance, GPT-4 and Claude 3 can solve 67.0% and 84.9% of function-level Python programming tasks in a zero-shot manner. Their impressive performance demonstrates the possibility of LLM-driven autonomous software development [36]. Hopefully, in the era of LLM, programming novices and even non-programmers will be able to build complex engineering projects through natural language. In this sense, natural language is one step closer to being the ultimate “programming language”. To explore this possibility, we propose a new software engineering task, called Natural Language to Code Repository (NL2Repo), with the goal of automatically generating an entire engineering code repository based on given natural language requirements.

However, automatically generating a real-world engineering project faces challenges due to the huge gap between the natural language description of requirements and the corresponding code repository. This gap is reflected in two aspects: (1) A real-world engineering project is typically accompanied by detailed natural language requirements to describe its numerous functional features. Unlike previous NL2Code tasks [48], natural language in NL2Repo will be longer and more complex. Although code LLMs have shown excellent performance in translating natural language to function, their instruction following capability and long-context length currently may not be sufficient for the NL2Repo task. (2) A target code repository typically exhibits high structural integrity, both in its directory tree and source code. Although code LLMs perform well in NL2Code, they excel more at generating simple standalone code snippets, such as line- or function-level code generation. This is primarily because auto-regressive language models are inherently ill-suited for modeling structured information [24].

To fill this gap, one straightforward idea is to decompose the complex NL2Repo task into sub-tasks via a sketch-based approach [33, 42, 47]. Earlier studies have proved the effectiveness of sketching in function-level program synthesis under formal specifications, like program sketching [39] and DreamCoder [12]. Nonetheless, those approaches fail in the NL2Repo task, as they do not support natural language input and hardly scale beyond function-level due to the large search space of candidate programs [5, 21, 29]. Notably, combining the sketch with code LLMs could complement both of these shortcomings, as probability-based models possess an inherent advantage at understanding natural language and handling large search space [5, 29]. Therefore, a thought-provoking question arises: how to leverage the insights of the sketch to empower code LLMs in better solving NL2Repo tasks?

To address this question, we propose CODES, a multi-layer Sketch-based framework for NL2Repo. CODES includes three modules: RepoSketcher, FileSketcher, and SketchFiller, which divide the NL2Repo into three phases. Specifically, RepoSketcher first generates the repository’s directory structure; for each filename in structure, FileSketcher then generates its file sketch which omits function body details; SketchFiller finally fills in each function body. The implementation of CODES framework involves two manners: prompt engineering and supervised fine-tuning. For the former, we directly utilize off-the-shelf code LLMs such as CodeLlama, DeepSeekCoder, and GPT-3.5, assigning them roles corresponding to the three modules. Regarding the latter, we fine-tune these base models to improve their capability to generate entire code repositories, thereby enhancing the overall performance of CODES.

To thoroughly evaluate the solutions for NL2Repo task, we conduct both benchmark-based automated and feedback-based manual evaluation. For benchmark-based evaluation, we craft a new benchmark, SketchEval, by collecting 19 latest GitHub repositories and devising a metric to assess the quality of generated repositories. For feedback-based evaluation, we develop a VSCode plugin for it and invite 30 participants to use it to implement

two real-world engineering projects. Extensive experiments on SketchEval and empirical studies have proved the effectiveness and practicality of CODES.

Overall, our contributions can be summarized as follows:

- We propose a new software engineering task, NL2Repo, with the goal of automatically generating an entire code repository based on given natural language requirements.
- We introduce CODES, a sketch-based framework to solve NL2Repo task. It divides NL2Repo into three phases and tackles them layer by layer.
- To comprehensively evaluate the NL2Repo task, we craft a benchmark SketchEval. It provides the evaluation of 19 real-world GitHub repositories and introduces a repository-oriented evaluation metric SketchBLEU. Furthermore, we develop a VSCode plugin and conduct empirical studies.
- Extensive experiments on SketchEval and empirical studies have proved the effectiveness and practicality of CODES. We have made our work publicly available¹.

2 CODES: A HIERARCHICAL FRAMEWORK FOR REPOSITORY GENERATION

2.1 Overview and Design Rationale

NL2Repo aims to generate *a functional code repository* from its *natural language descriptions*, with a particular focus on maintaining both syntactic and semantic correctness across the entire repository. Though the input and output of a NL2Repo task are similar to a conventional NL2Code task [48], we consider it as a new task instead of a direct extension. Even though both NL2Repo and NL2Code adopt natural language as their inputs. The inputs of NL2Repo tend to be much longer with diverse sources of information. It typically includes a project title, functional description, features, dependencies, usage examples, FAQs, etc. Also, the functional descriptions in NL2Repo tend to be much more high-level compared to conventional NL2Code tasks. The expected size of the produced code is way larger than its descriptions. NL2Repo targets repository generation instead of code segments. It proposes new challenges in designing file structures, modules, and interfaces besides conventional code generation. In this paper, we consider a project's README.md file as its natural language input². With this specific setting, we propose CODES, a multi-layer sketch framework for NL2Repo.

CODES consists of three modules: RepoSketcher, FileSketcher, and SketchFiller, as Fig. 1 shows. Given a ①README.md, RepoSketcher first aims to generate a ②Repository Sketch. It includes the names of all directories and files in the form of a tree. Next, FileSketcher generates a ③File Sketch for each file in the generated repository sketch. The file sketch is designed to capture both local and global dependencies, including import statements, class definitions, and inter-file references. Specifically, when generating each FileSketch, the module analyzes the repository-wide context to: (1) determine necessary import statements by examining class and function references across files, (2) establish proper module interfaces through carefully structured class and function signatures, and (3) maintain consistency in attribute and method names across related files. This comprehensive dependency analysis ensures that the generated code maintains both syntactic correctness (e.g., no undefined references) and semantic coherence (e.g., consistent API usage) at the repository level. Finally, SketchFiller is responsible for filling in the ④Function Body for each function definition. The arrows in Fig. 1 show the information flow of CODES. It can be summarized as follows ②=RepoSketcher(①), ③=FileSketcher(①,②), and ④=SketchFiller(①,②,③). Noted that, the FileSketcher and SketchFiller aggregate information from previous stages. This design enables CODES to progressively refine the sketch layer by layer without losing high-level task information. The three modules of CODES are designed to collaborate seamlessly to address the requirements of the complex project.

¹<https://github.com/NL2Code/CodeS>

²README.md here has a broadened interpretation, where developers can state their requirements using NL. In training and inference, we use processed README.md as the inputs, where contents unrelated to requirements are removed.

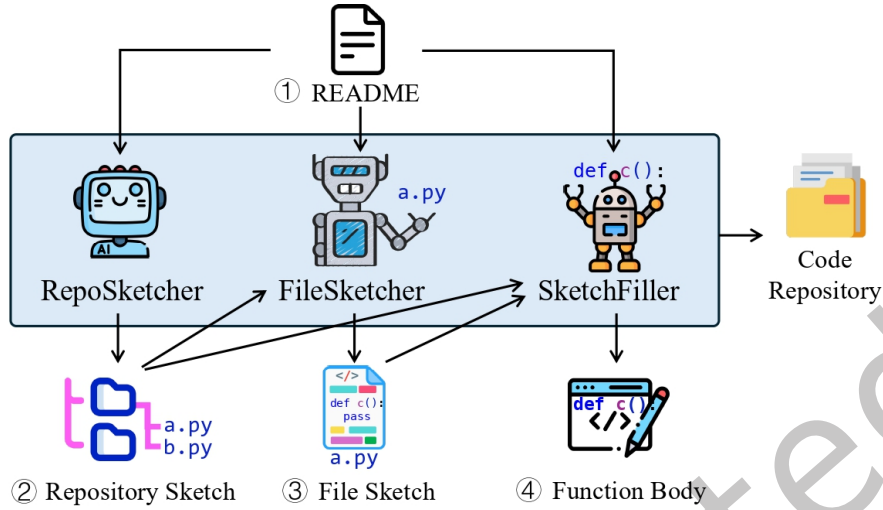


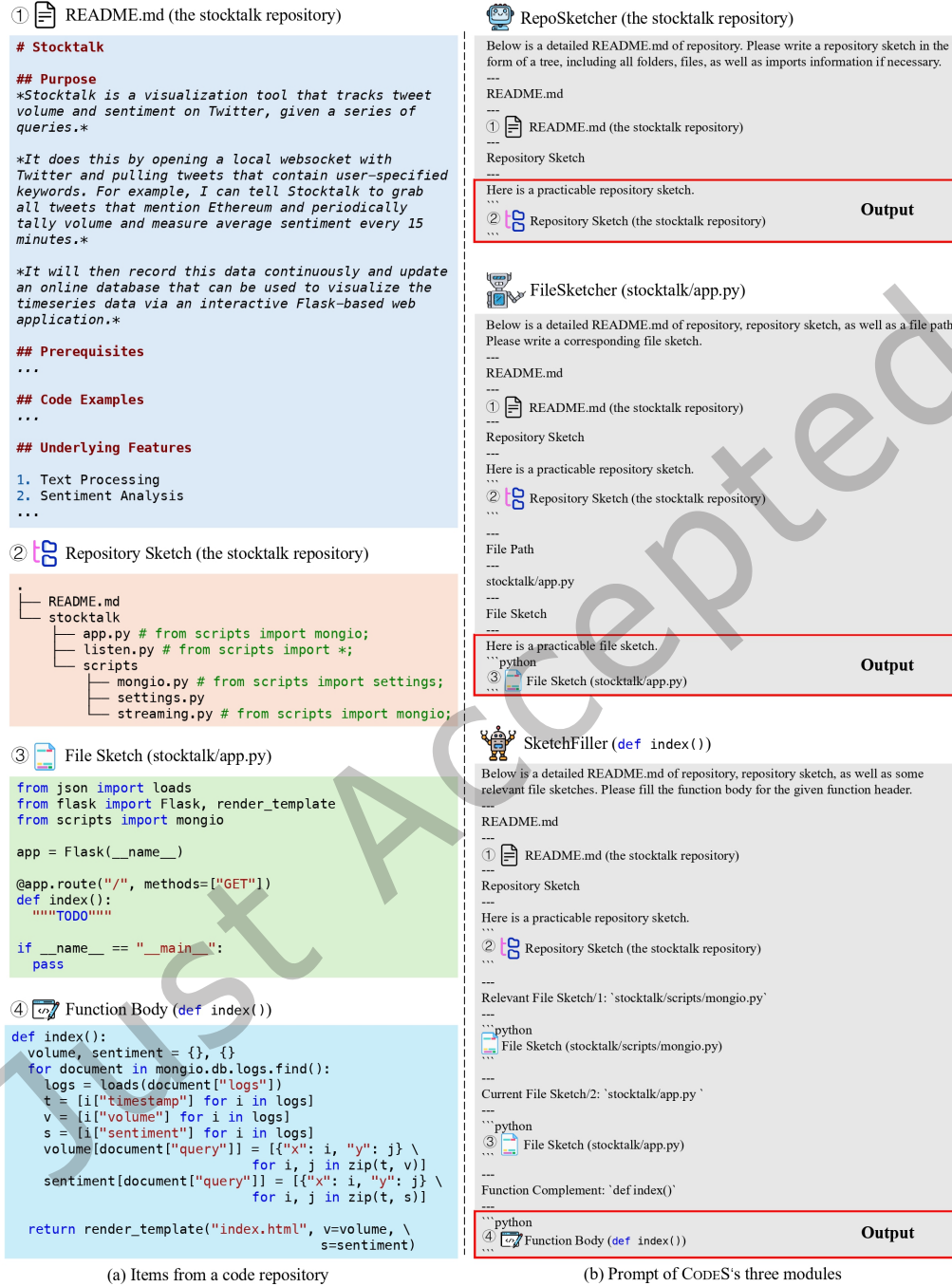
Fig. 1. Overview of our proposed framework CODES.

2.2 Technical Components

To address the challenges of maintaining syntactic and semantic correctness across the repository, we implement the CODES framework with careful consideration of cross-file dependencies and global context, namely prompt engineering (PE) and supervised fine-tuning (SFT). For the former, we prompt the off-the-shelf code LLMs, such as StarCoder2 [27], CodeLlama [37], DeepSeekCoder [14], and OpenAI’s GPT-3.5 [32] to function as the three modules RepoSketcher, FileSketcher and SketchFiller. For the latter, we leverage instruction fine-tuning to adapt base models to fulfill the functions of the three modules. Both of the above implementation choices share the same workflow of CODES, with specified inputs and outputs of each module. The specification of inputs and outputs serves as the foundation for the construction of both the inference prompts and fine-tuning data. Exemplary prompt templates of CODES are presented in Fig. 2. We proceed to elaborate on each module’s design, explaining their input and output format.

Fig. 2 (a) presents four items used in the inputs and outputs of three modules, including ①README.md, ②Repository Sketch, ③File Sketch and ④Function Body. A ①README.md contains the project title, description, features, installation, usage examples, table of contents, change logs, dependencies, FAQs, etc. We only reserve the first five parts to prevent irrelevant or redundant information from affecting LLMs. A ②Repository Sketch formats like a directory tree generated by the Linux tree command, but additionally attached with cross-file imports as annotations after each code file entry. A ③File Sketch for each code file in the repository formats as a Python snippet where the bodies of all the functions are replaced with a placeholder statement. A ④Function Body is a function-level code snippet that completes the specified target function signature. The above four items are used to compose the inputs and outputs of CODES’ modules.

Fig. 2 (b) lists the prompts used in each module, depicting their inputs and outputs. The inputs of all the modules contain a brief description of the task and necessary items selected from ①README.md, ②Repository Sketch and ③File Sketch. The outputs contain a prefixed statement identifying the response type, and a corresponding item of ②Repository Sketch, ③File Sketch or ④Function Body respectively for the module of RepoSketcher, FileSketcher, and SketchFiller. Specifically, RepoSketcher accepts a ①README.md and outputs a ②Repository Sketch, while FileSketcher accepts a ①README.md a ②Repository Sketch and the target file path to generate,

Fig. 2. Prompt templates of CODES's three modules on a repository (<https://github.com/anfederico/stocktalk>).

outputting a corresponding ③File Sketch. For SketchFiller, the input contains a ①README.md, a ②Repository Sketch, optional relevant ③File Sketch, current ③File Sketch and the target function signature to complete. Among them, the relevant ③File Sketch corresponds to those dependent code files that the current file may import, hence they are included in the input for SketchFiller’s reference; the current ③File Sketch presents both the context and location of the target function, with the body of the target function set as a “TODO” placeholder statement while that of the other functions set as a “pass” statement. Finally, SketchFiller outputs a corresponding ④Function Body to complete the target function. These above three modules are connected, with the output of one serving as the input of the next, collectively forming the CODES framework.

3 SKETCHEVAL: A BENCHMARK FOR NL2REPO

We craft a benchmark SketchEval to evaluate the NL2Repo tasks for Python. SketchEval includes: (1) a dataset of 19 real-world repositories of varying complexity, which are delicately selected from the latest open-source GitHub projects. (2) a metric to measure the similarity between two repositories, in terms of structures and semantics.

3.1 Dataset for Evaluation

To collect the evaluation repositories, we follow three steps: crawling, filtering, and grouping.

- **Firstly**, we extensively crawl open-source repositories via GitHub API. From GitHub, we request around 300 most-starred repositories³ with Python as the main language from August 2023 onwards⁴.
- **Secondly**, we refine the crawled repository set by filtering out repositories based on their type, quality, and topic. (1) for types, we rule out those framework- or library-oriented repositories, mainly focusing on standalone projects. (2) for quality, we select those repositories with a well-organized README and well-constructed source code. For each repository, we require that its README document contains a meta description and a detailed feature description. The descriptions are necessary to declare the goal of a repository. we also check the validity of the source code by manually running the repository or reading the code. (3) for topics, we try our best to cover diverse areas of repositories (e.g. AI, cyber security, shell tools, etc.).
- **Thirdly**, we group the selected repositories into different difficulty levels based on the scale of a repository. The repositories are grouped into three levels: repositories of *Hard* level should contain > 10 Python files or > 2500 Python code lines; Repositories of *Medium* level should contain > 5 Python files or > 500 Python code lines; Other repositories are classified as *Easy* level.

As a result, we totally collect a set of 19 repositories⁵ as *reference repositories*, including 5 easy ones, 8 medium ones, and 6 hard ones. The information of them is listed in Table 1. For the three modules of CODES, these 19 repositories provide 19, 189, and 1,166 evaluation instances, respectively.

To further quantify the human effort required to construct the benchmark, we report the number of lines and tokens in each repository’s README.md file (see Table 1), with tokens computed using the BERT-base-uncased tokenizer. On average, the README files contain 111 lines and 1,509 tokens, reflecting the practical workload involved in preparing natural language inputs for CodeS.

3.2 Metric Definition

3.2.1 Overview. To evaluate repository generation performance, we design a metric, namely SketchBLEU, which can calculate a repository-level similarity between the generated repository with the reference one. We follow

³https://api.github.com/search/repositories?q=language:Python+created:>2023-08-01&sort=stars&order=desc&per_page=100 &page={page}

⁴We choose this date to prevent data leakage in our base models.

⁵<https://github.com/NL2Code/CodeS/tree/main/validation/repos>

Table 1. Meta information of the selected repositories in SketchEval. #F. and #L. denote the number of Python files and code lines in the repository. #MD Lines and #MD Tokens represent the number of lines and tokens (calculated using BERT-base-uncased tokenizer) in the repository’s README.md file.

Difficulty	Repository Name	Stars	Created	#F.	#L.	#MD Lines	#MD Tokens
Easy	CVE-2023-44487	205	2023-10-10	1	319	40	497
	EVM_inscription	227	2023-12-18	3	164	48	624
	pitch-visualizer	192	2023-12-05	2	315	70	857
	smol-podcaster	261	2023-08-08	1	277	96	994
	web.Monitor	118	2023-09-09	1	237	153	2195
Medium	django-tui	212	2023-08-23	6	873	59	636
	easier-docker	151	2023-11-23	10	274	60	826
	epubhv	444	2023-09-04	7	958	96	1023
	every-breath-you-take	541	2023-09-16	5	1497	43	518
	fastui-chat	190	2023-12-17	8	367	94	836
	kanban-python	228	2023-11-11	10	1945	211	3355
	libgen_to_txt	207	2023-10-16	9	570	82	1337
	van-gonography	384	2023-11-13	4	1030	176	2744
Hard	EasyLiterature	174	2023-10-09	13	1564	173	3275
	flameshow	956	2023-09-24	19	2106	83	960
	mactop	132	2023-12-05	43	3639	122	916
	pygraft	611	2023-09-07	12	3882	185	2429
	pyobd	709	2023-08-18	22	12220	192	3027
	sim-web-visualizer	223	2023-08-10	18	4870	117	1625

the idea of CodeBLEU [35], a metric that works effectively on function-level code evaluation. CodeBLEU is a weighted combination of four parts including n-gram BLEU, weighted n-gram BLEU, syntactic tree match, and semantic data-flow match. Based on CodeBLEU, we propose SketchBLEU by adapting the four parts of CodeBLEU to repository-level code evaluation, which can be defined as:

$$\text{SketchBLEU} = \alpha \cdot \text{BLEU}' + \beta \cdot \text{BLEU}'_{\text{weight}} + \gamma \cdot \text{Match}_{\text{struc}} + \delta \cdot \text{Match}'_{\text{df}} \quad (1)$$

where BLEU' and $\text{BLEU}'_{\text{weight}}$ respectively denote the standard n-gram BLEU precision and the weighted n-gram BLEU precision that is calculated based on the concatenated source code of the repository; $\text{Match}_{\text{struc}}$ denotes a n-hop tree match ($n = 3$ in practice) of the *structural tree*, which is constructed by concatenating the directory tree and the abstract syntax trees (ASTs) of Python code files in the repository; $\text{Match}'_{\text{df}}$ denotes the semantic dataflow match between repositories, calculated by matching function-level dataflows between repositories via maximum weighted bipartite matching. In our experiment, we assign equal weights to the four sub-parts of the metric, with $\alpha = \beta = \gamma = \delta = 0.25$. To transfer function-level CodeBLEU to repository-level SketchBLEU, we make efforts to adjust the four sub-parts of the original metric. For instance, we adopt a less sensitive brevity penalty, because repositories implementing the same requirements can vary greatly in size.

We adopt SketchBLEU to measure the generated repositories’s quality instead of execution-based metrics (e.g., compilation error and build-rates) for the challenges in running environment setup. SketchEval collects real-world repositories across different domains. For some of them, it is hard to build a compile or execution environment, e.g., the every-breath-you-take repository⁶ has specific hardware requirements.

⁶<https://github.com/kieranabrennan/every-breath-you-take>

3.2.2 A Detailed Specification of SketchBLEU. Resembling CodeBLEU [35], SketchBLEU is a weighted combination of four sub-parts ($BLEU'$, $BLEU'_{weight}$, $Match_{struc}$ and $Match'_{df}$), in place of the original four sub-parts of CodeBLEU ($BLEU$, $BLEU_{weight}$, $Match_{ast}$ and $Match_{df}$). To transfer the function-level metric CodeBLEU to a new repository-level metric SketchBLEU, we make several modification to the original sub-parts of the CodeBLEU metric. Specifically, we make four main modifications:

- **Firstly**, we replace the single code snippet used to calculate the original $BLEU$ and $BLEU_{weight}$ with a concatenation of all the code files in the repository.
- **Secondly**, we replace the BLEU brevity penalty with a less sensitive one because the size of repositories designed to meet the same requirements can differ markedly. The modified brevity penalty can be formalized as:

$$BP'_r = \begin{cases} 1 & \text{if } 2 \cdot c > r \\ \frac{1}{1 + \ln r - \ln 2 \cdot c} & \text{if } 2 \cdot c \leq r \end{cases} \quad (2)$$

- **Thirdly**, we replace the ASTs used to calculate the original $Match_{ast}$ with a structural tree combining both directory structure and source code syntax, thus deriving $Match_{struc}$. Also, instead of counting the fully matched sub-trees as the original $Match_{ast}$ does, we extract n-hop sub-trees, truncating the sub-trees to limit its depth. The reason behind this modification is that, compared to function-level codes, repository have a multi-layered structure, hard to be evaluated based on a full-depth sub-tree.
- **Fourthly**, we extend the original function-to-function dataflow match metric $Match_{df}$ to a repository-to-repository one, namely $Match'_{df}$:

$$Match'_{df} = \begin{cases} BP'_{\frac{|Hyp|}{|Ref|}} \cdot \frac{MWB}{|Hyp|} & \text{if } |Ref| > |Hyp| \\ BP'_{\frac{|Ref|}{|Hyp|}} \cdot \frac{MWB}{|Ref|} & \text{if } |Ref| \leq |Hyp| \end{cases} \quad (3)$$

where MWBM denotes the maximum bipartite weighted matching of a complete bipartite graph with (Ref, Hyp) as the partition and $Match_{df}$ as the weight function; Ref and Hyp respectively correspond to functions in the reference repository and the prediction repository. We also use the modified brevity penalty BP' to measure the size gap between the function sets of the reference repository and the prediction repository.

For more details, please refer to the implementation of SketchBLEU, which can be found in our open-source repository⁷.

4 RESEARCH QUESTIONS

We aim to comprehensively evaluate our framework CODES under the NL2Repo task by answering the following three Research Questions (RQs):

- RQ1: How does CODES based on multi-layer sketch perform on the NL2Repo task?
- RQ2: What factors influence the performance of CODES?
- RQ3: Does CODES hold practical potential in real-world NL2Repo tasks?

We answer RQ1 by evaluating CODES against all its baselines on our constructed benchmark SketchEval. Regarding RQ2, we perform a detailed analysis for CODES from the perspectives of base model and instruction data. In response to RQ3, we develop a VSCode plugin for CODES and invite 30 participants to develop two projects using it.

⁷https://github.com/NL2Code/CodeS/tree/main/validation/evaluation_scripts

5 EXPERIMENTAL SETUP

5.1 100 Code Repositories for Supervised Fine-Tuning

To adapt base models for the NL2Repo task, we perform supervised fine-tuning based on a collection of 100 code repositories. To obtain these repositories, we first crawl Python repositories created before August 1, 2023 from GitHub. Choosing this creation date as a filtering criterion aims to prevent data leakage into our newly constructed evaluation benchmark SketchEval, thus ensuring the fairness of our experiments. After that, we further exclude repositories with less than 100 stars to ensure high-quality training data.

Based on the filtered repositories, we manually select 100 high-quality Python ones⁸. For each of these repositories, we extract its four parts: ①README.md, ②Repository Sketch, ③File Sketch, and ④FunctionBody. One concrete example for these four parts is shown in Fig. 2 (a). Based on the collected repositories, we construct a total of 7,806 instruction data for CODES's three modules: 100 for RepoSketcher, 1,191 for FileSketcher, and 6,515 for SketchFiller. Fig. 2 showcases the prompt templates for CODES's three modules.

5.2 CODES and Its Comparative Methods

We implement the prompt engineering (PE) and supervised fine-tuning (SFT) versions of CODES based on multiple base models⁹, including CodeLlama-Instruct 7&13&34B [37], DeepSeekCoder-Instruct 7&33B [14], StarCoder2 3&7&15B [27], and OpenAI's GPT-3.5-turbo [32]. Note that all GPT-3.5-based methods have only PE versions. The baselines of CODES include two categories:

- **Vanilla:** Our contributions lie in producing a complex code repository by utilizing a three-layer sketch. Therefore, a primary baseline for CODES is the method without any sketching, which means leveraging code LLMs to directly generate a repository. We implement the PE and SFT versions for it based on GPT-3.5 and CodeLlama.
- **Agent-based Methods:** They were initially proposed to accomplish a specific objective through the collaborative working of multiple LLMs. In the NL2Repo task with the objective of generating a repository, we reproduce three agent-based methods based on GPT-3.5 including ChatDev [34], AutoGPT [43], and AgentGPT [2]. Those agent-based methods cannot directly solve NL2Repo, so we adapt them via prompt engineering. All methods are prohibited from accessing any tools. Following the design of ChatDev [34], they all include five agents (CEO, CTO, Programmer, Reviewer, Tester) and have same system and instance prompts, differing in agent-communication mechanisms (i.e., the conversational and feedback framework).

5.3 Implementation Details

We use DeepSpeed¹⁰ and Llama-Factory [55] to fine-tune base models. During the instruction fine-tuning process of CODES, the batch size is set to 8 per GPU card, gradient accumulation steps to 8, and saving steps to 25. We set the learning rate to 5e-5 with cosine decay, and enable fp16 to accelerate training. AdamW [26] is used to optimize the parameters with $\beta=(0.9, 0.99)$ and $\epsilon=1e-8$. Also, we employ Rotary Position Embedding (ROPE) [40] to enhance the long-context modeling capability of the model. For inference, we use greedy sampling across all our experiments on SketchEval.

During the empirical study, we deploy CODES using vLLM [20] and provide API services for the VSCode plugin. When requesting the API of CODES, we use nucleus sampling with the temperature of 0.2, top_p of 0.9, frequency_penalty of 0.35, and presence_penalty of 0.25. Regarding the instruction data, we use the tree

⁸<https://github.com/NL2Code/CodeS/tree/main/repos>

⁹The four base models were pre-trained with GitHub data before February 2023, September 2023, August 2023, and September 2021, respectively.

¹⁰<https://github.com/microsoft/DeepSpeed>

Table 2. Performance of CODES and its baselines on SketchEval. E., M., and H. stand for easy, medium, and hard. $BLEU'$ (B.), $BLEU'_{weight}$ (B.W.), $Match_{struc}$ (M.S.), and $Match_{df}$ (M.D.) denote the four parts of SketchBLEU. DeepSeekCoder is abbreviated to DSCoder.

Method	Base Model	SketchEval							
		E.(5)	M.(8)	H.(6)	All(19)	All(19)			
		SketchBLEU				B.	B.W.	M.S.	M.D.
Prompt Engineering (PE)									
Vanilla	GPT-3.5	19.23	14.56	12.82	15.24	14.25	14.11	18.41	14.19
	CodeLlama 7B	18.45	15.63	9.54	14.45	13.55	13.74	17.36	13.15
	CodeLlama 13B	20.63	15.48	12.33	15.84	14.88	15.59	19.42	13.47
	CodeLlama 34B	23.52	17.36	11.12	17.01	15.63	16.73	22.57	13.11
ChatDev	GPT-3.5	56.13	46.93	31.17	45.13	41.06	42.44	52.35	44.65
	DSCoder 33B	58.01	53.57	36.63	49.39	45.32	44.53	55.74	51.97
	CodeLlama 34B	56.43	45.84	32.46	44.4	42.75	42.22	52.09	40.54
AutoGPT	GPT-3.5	55.53	45.24	31.76	43.69	41.34	43.86	51.52	38.04
	DSCoder 33B	57.45	53.64	34.70	48.66	44.57	44.46	54.99	50.62
	CodeLlama 34B	55.56	46.57	32.04	44.35	42.36	44.02	51.26	39.76
AgentGPT	GPT-3.5	53.35	44.14	30.13	42.14	40.01	39.64	53.34	35.57
	DSCoder 33B	55.57	51.65	33.14	46.84	43.86	43.02	55.26	45.22
	CodeLlama 34B	53.73	45.25	31.56	43.16	40.35	40.26	53.68	38.35
CODES	GPT-3.5	52.41	52.96	36.54	47.63	44.27	44.58	57.21	44.44
	DSCoder 7B	50.15	52.24	34.29	46.02	42.56	43.00	46.61	52.91
	DSCoder 33B	58.47	57.36	39.26	51.94	46.14	48.64	59.33	53.65
	StarCoder2 3B	28.87	28.24	14.53	24.08	20.56	20.24	32.51	23.01
	StarCoder2 7B	32.45	29.95	15.24	25.96	23.24	24.94	31.23	24.43
	StarCoder2 15B	35.13	32.54	18.35	28.74	25.35	27.75	35.84	26.02
	CodeLlama 7B	50.63	51.52	33.47	45.59	42.45	43.03	45.58	51.30
	CodeLlama 13B	51.44	52.15	34.64	46.43	41.14	42.52	46.57	55.49
	CodeLlama 34B	56.14	54.50	36.74	49.32	43.55	44.51	52.73	56.49
Supervised Fine-Tuning (SFT)									
Vanilla	CodeLlama 7B	26.35	22.53	21.59	23.24	22.96	22.46	25.53	22.01
	CodeLlama 13B	30.14	26.56	22.21	26.13	25.31	26.23	28.87	24.11
	CodeLlama 34B	35.63	29.44	27.48	30.45	28.46	28.78	33.77	30.79
CODES	DSCoder 7B	57.47	57.78	52.86	56.14	54.35	54.75	65.66	49.80
	DSCoder 33B	65.05	63.36	61.53	63.23	60.44	62.41	68.35	61.72
	StarCoder2 3B	35.56	32.03	24.57	30.60	26.36	28.76	38.52	28.76
	StarCoder2 7B	48.23	46.31	39.78	44.75	41.45	42.32	42.65	52.58
	StarCoder2 15B	57.25	55.98	47.56	53.66	49.99	51.46	58.35	54.84
	CodeLlama 7B	55.96	56.11	50.78	54.39	50.49	51.81	63.92	49.35
	CodeLlama 13B	58.24	57.25	56.35	57.23	54.24	53.26	58.25	63.17
	CodeLlama 34B	65.14	64.24	60.56	63.31	57.46	59.25	71.55	64.98

command in Linux to extract repository sketch from a code repository. We employ the `ast` library¹¹ for extracting the file sketch and function bodies from a Python file. We format all the crawled source code using the `black` library¹². The version of OpenAI’s GPT-3.5 used in our experiments is the `gpt-3.5-turbo-0613`¹³.

6 RQ1: THE EFFECTIVENESS OF CODES

This section will initially evaluate the performance of CODES and its baselines on SketchEval, followed by delving into CODES’s three modules.

6.1 Performance Analysis of CODES

Table 2 shows the performance of CODES and its baselines on SketchEval. Compared to the baseline Vanilla without sketching, CODES with multi-layer sketch exhibits significant performance improvements. This underscores the necessity of incorporating the multi-layer sketch into code LLMs. Additionally, CODES outperforms three agent-based baselines in medium and hard repositories, while performing comparably in easy ones. This is because easy repositories do not heavily rely on the multi-layer sketch. Among the baselines, larger models like CodeLlama 34B and DeepSeekCoder 33B demonstrate stronger performance, particularly in handling complex repository generation tasks, which highlights the importance of model capacity. However, in hard repositories with complex directory structures, these baselines still fall short because they do not explicitly decompose repositories’ complexity, highlighting the effectiveness of CODES’s multi-layer sketch approach. Furthermore, we observe that CODES’s SFT versions consistently surpass the PE ones. As an example, after supervised fine-tuning, CODES based on CodeLlama 7B, 13B, and 34B increases SketchBLEU scores from 45.59%, 46.43%, 49.32% to 54.39%, 57.23%, 63.31%, respectively. This observation emphasizes the necessity of supervised fine-tuning on our crafted instruction data in Section 5.1. Besides, for all PE versions of CODES, there exists a notable performance drop as the difficulty of the repositories increases. However, the SFT versions show a less pronounced decline, further underscoring the value of instruction fine-tuning.

In addition, we also compare the number of files/lines in predicted code repositories versus that in reference ones. The results of CODES and ChatDev are displayed in Fig. 3. Regardless of the number of files or lines, CODES’s predictions are consistently closer to the reference, compared to that of ChatDev. This suggests that CODES is able to predict the number of files and lines of the target repository more accurately. Interestingly, regardless of the complexity of requirements, the repositories generated by ChatDev always contain no more than 8 files or 842 lines. CODES, in contrast, supports up to 25 files and 7,334 lines. This indicates that CODES, with its multi-layer sketch, is able to mitigate the shortcomings of code LLMs in modeling long contexts.

6.2 Performance Analysis of Three Modules of CODES

Table 3 shows the performance of CODES’s three modules on code repositories of varying difficulty. In this experiment, all input items of each module are derived from the reference repository. For the PE version of CODES, as the difficulty increases, the most significant performance drop occurs during the first of the three phases. For example, CodeLlama 13B experiences a 28% decrease in BLEU score from easy (77%) to hard levels (49%). This suggests that the primary obstacle for the PE version of CODES lies in the first (generating repository sketch) phase when addressing the complex repository generation. Upon applying SFT, CODES demonstrates a significant performance improvement in the first phase. Such findings further highlight the importance of instruction fine-tuning. Table 4 offers a statistical analysis of the outputs from CODES’s three modules, drawing conclusions similar to those in Table 3. For instance, for the PE version, the average number of files in the

¹¹<https://docs.python.org/3/library/ast.html>

¹²<https://pypi.org/project/black>

¹³<https://platform.openai.com/docs/models/gpt-3-5-turbo>

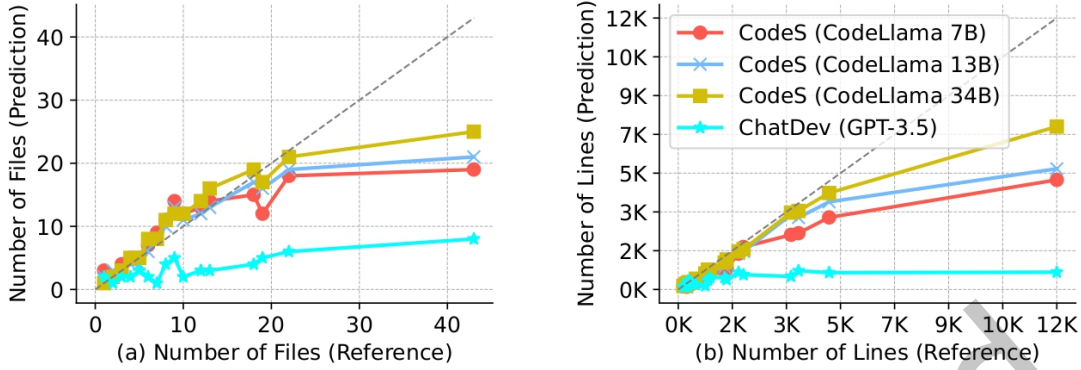


Fig. 3. Comparison between predicted and the reference repositories by number of files (a) and lines (b).

Table 3. Performance of RepoSketcher (phase 1), FileSketcher (phase 2), and SketchFiller (phase 3) of CODES on SketchEval’s code repositories of varying difficulty. E., M., and H. denotes easy, medium, and hard.

Method	Base Model	Phase 1				Phase 2				Phase 3				
		Repo. Sketch				File Sketch				Fun. Body				
		BLEU				BLEU				CodeBLEU				
		E.	M.	H.	All	E.	M.	H.	All	E.	M.	H.	All	
Prompt Engineering (PE)														
CODES	GPT-3.5	75	69	45	63	47	45	39	45	66	64	62	64	
	CodeLlama 7B	67	64	46	59	45	38	38	40	65	63	64	64	
	CodeLlama 13B	77	72	49	66	45	43	44	44	71	67	67	68	
	CodeLlama 34B	84	81	60	75	56	53	50	53	76	70	71	72	
Supervised Fine-Tuning (SFT)														
CODES	CodeLlama 7B	72	70	62	68	55	50	52	52	74	71	68	71	
	CodeLlama 13B	81	81	65	76	62	57	56	58	80	76	79	78	
	CodeLlama 34B	88	84	68	80	64	61	58	61	83	81	79	81	

generated repository sketch for CodeLlama 34B is 5, while the reference is 10. However, after SFT, it reaches an average of 11 files, further demonstrating the potential of our curated data.

7 RQ2: INFLUENCING FACTORS OF CODES

In this section, we explore the factors that influence the performance of CODES, focusing on two critical aspects: the base model and instruction data. Specifically, we first investigate the impact of different sizes of various base models on both the PE and SFT versions. Then, we study how dataset scale and instruction format affect the effectiveness of SFT.

7.1 Base Model

Fig. 4 (a) illustrates the SketchBLEU of CODES’s PE and SFT versions across various base models of different sizes. Generally, a larger base model yields a higher SketchBLEU for both the PE and SFT versions, and SFT consistently outperforms PE by an average improvement of 13.15% in SketchBLEU. Furthermore, the effectiveness

Table 4. Summary statistics for the outputs of RepoSketcher, FileSketcher, and SketchFiller. #D., #F., and #I. are the number of directories, files, and imports in repository sketch; #c. and #f. represent class and function counts in file sketch; #API and #L. denote API and line counts in function body.

Method	Base Model	Phase 1 Repo. Sketch			Phase 2 File Sketch		Phase 3 Fun. Body	
		#D.	#F.	#I.	#c	#f.	#API	#L.
Reference	-	4	10	7	2	5	6	13
Prompt Engineering (PE)								
CODES	GPT-3.5	2	3	2	1	4	5	10
	CodeLlama 7B	1	4	2	0	3	3	12
	CodeLlama 13B	2	4	4	1	3	4	13
	CodeLlama 34B	3	5	5	1	4	6	12
Supervised Fine-Tuning (SFT)								
CODES	CodeLlama 7B	2	10	4	1	3	4	13
	CodeLlama 13B	3	10	5	2	5	4	12
	CodeLlama 34B	4	11	8	1	6	6	14

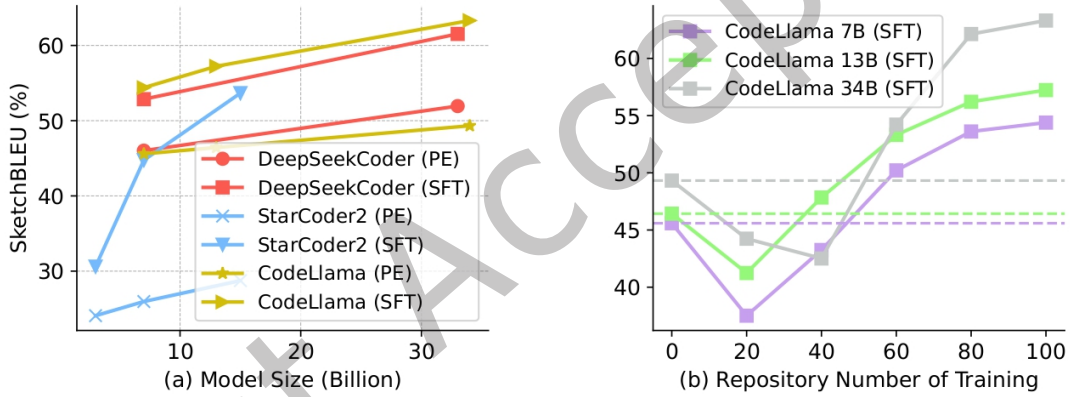


Fig. 4. SketchBLEU of CODES across different settings on SketchEval.

of instruction fine-tuning appears to become increasingly significant as the model size grows. For example, for CodeLlama with a size of 7B, the improvement on SketchBLEU brought by SFT compared with PE is 8.89%, while for CodeLlama of larger size, such as 33B, the improvement in SketchBLEU jumps to 13.99%. This is mainly because the effectiveness of SFT depends largely on the knowledge encoded in LLMs [51]. As the model size increases, its ability to encode and leverage intricate knowledge also expands, consequently boosting the effectiveness of instruction fine-tuning that unlocks the potential of LLMs for NL2Repo tasks.

7.2 Instruction Data

7.2.1 Data scale. Fig. 4 (b) shows the performance of CODES on SketchEval as the scale of the instruction dataset expands. It demonstrates that SFT becomes effective when the number of repositories used for fine-tuning surpasses a specific threshold. As can be seen from Fig. 4 (b), this threshold for CodeLlama series models is 60. In

Table 5. Ablation study of CODES.

Method	SketchEval			
	Easy (5)	Middle (8)	Hard (6)	All (19)
CODES (CodeLlama 13B)	58.24	57.25	56.35	57.23
+Table of Contents	57.56	55.00	50.53	54.26
+Change Logs	56.14	53.24	51.03	53.31
+Dependencies	58.02	54.89	49.42	53.99
+FAQs	57.47	56.35	50.46	54.78
-Import	56.35	50.01	44.04	49.79
-Tree Format	56.63	53.45	48.87	52.84
-TODO	53.15	49.13	42.41	48.07
-Relevant File Sketch	57.94	52.56	43.57	51.14

other words, the instruction fine-tuning on CodeLlama outperforms prompt engineering (i.e., #Repositories = 0) only when the number of repositories for fine-tuning equals or exceeds 60. When the number of repositories falls below the specific threshold, SFT results in a decline in performance, which is particularly noticeable with CodeLlama 7B. This decline is mainly attributed to the over-fitting caused by a relatively small fine-tuning dataset scale. When the number of repositories exceeds the specific threshold, SketchBLEU tends to increase as more repositories are involved in fine-tuning. However, this increase is limited; there is a tendency to reach a plateau when the number of training repositories reaches or exceeds 80. These observations further validate our decision to use 100 repositories for fine-tuning, which achieves a cost-effective balance between performance and fine-tuning cost.

7.2.2 Instruction Format. Table 5 exhibits the results of the ablation study conducted on the components of CODES. Specifically, we propose eight variants of CODES’s prompt by adding (+) new components or removing (-) existing components. For ①README.md, CODES employs a data pre-processing process to filter out irrelevant parts, including the table of contents, change logs, dependencies, and FAQs. In Table 5, we separately add each of these four filtered parts to analyze their influence on the performance of CODES. Regarding ②Repository Sketch, it includes relevant “import” information appended to each filename and adopts a tree format to organize the repository sketch. To evaluate the effect of these two components, we (1) remove the import information (-Import) and (2) replace the tree format with a multi-layer parenthesis to describe the file structure (-Tree Format). As for ③File Sketch, it utilizes a “TODO” keyword as a placeholder to enable the model to identify completion places, and we also remove it (-TODO) to analyze CODES. Additionally, in the prompt for the SketchFiller module, CODES includes a part for relevant file sketch to capture cross-file information. In Table 5, we remove this part (-Relevant File Sketch) to investigate the impact of cross-file information on CODES.

Table 5 shows that CODES consistently achieves the highest SketchBLEU, indicating the superiority of its instruction fine-tuning strategy. Additionally, it can be observed that this superiority becomes more significant as task complexity increases. For example, after removing the import information, the decrease in SketchBLEU on the easy task is 1.89%, while that value on the hard task is 12.31%. The same phenomenon can be observed when Relevant File Sketch is removed. This discrepancy is mainly attributed to the augmented complexity of file dependencies in harder tasks. For all added components, we observe that meaningless noise information in ①README.md degrades the model’s performance. Therefore, we can conclude that the pre-processing of ①README.md is a necessary step for enhancing the performance for NL2Repo tasks. Among the existing components in CODES, the removal of the “TODO” keyword results in the largest decrease in SketchBLEU,

Table 6. SketchBLEU of CODES (SFT) using inter-file topological ordering.

Method	SketchEval			
	Easy (5)	Middle (8)	Hard (6)	All (19)
CODES (CodeLlama 7B)	55.96	56.11	50.78	54.39
+Ordered Generation	55.67	56.32	51.00	54.47
CODES (CodeLlama 13B)	58.24	57.25	56.35	57.23
+Ordered Generation	58.15	57.06	58.46	57.79
CODES (CodeLlama 34B)	65.14	64.24	60.56	63.31
+Ordered Generation	64.86	64.13	61.63	63.53

Table 7. Repository Sketch’s BLEU of RepoSketcher (fine-tuned on Python) on different languages.

Evaluation Languages	RepoSketcher of CODES (CodeLlama 13B)	
	Prompt Engineering	Supervised Fine-Tuning on Python
Java	46.25	57.36
C++	36.68	50.14
Go	62.88	71.57

emphasizing the pivotal role of this keyword in enabling LLMs to identify the appropriate generation context. Regarding the removal of the tree format, the pre-trained model is more familiar with this format instead of others such as multi-layer parenthesis since the tree format is a widely adopted pattern in practice. Therefore, the fine-tuning based on the tree format is more effective since it is aligned with the knowledge in LLMs.

7.3 The Designs of CODES

7.3.1 The Impact of The Generation Order of Files on CODES. CODES proposed in Section 2.2 does not distinguish the generation order of repository files. We would like to generate files in accordance with the sequence determined by their invocation relationships. Taking ② of Fig. 2 (a) as an example, we can observe that “mongio.py” invokes “settings.py”. So, we generate “settings.py” before generating “mongio.py”; the same principle applies to the remaining files. Results in Table 6 indicate that ordered generation results in a minor performance decline on easy repositories but yields a slight performance improvement on hard repositories. Through empirical analysis, we conclude this is reasonable, as more hard repositories often involve intricate invocation relationships between files, which are less prevalent in easier ones.

7.3.2 Cross-Language Generalization of CODES. We are intrigued to explore whether CODES trained on Python repositories can be generalized to other languages. Therefore, we generate repository sketch for Java, C++, and Go projects using CODES’s RepoSketcher¹⁴. In Table 7, we evaluate two versions of RepoSketcher: prompt engineering and supervised fine-tuning. We observe that fine-tuning on Python enhances RepoSketcher’s performance on other languages as well. This indicates that fine-tuning CODES on a single language can further unlock the potential of LLMs to perform NL2Repo across multiple languages [49].

¹⁴We craft an evaluation set of 19 repositories for each language, following the same process outlined in Section 3.1. These repositories can be seen at <https://github.com/NL2Code/CodeS/tree/main/validation/multilingual-repos>.

8 RQ3: THE PRACTICALITY OF CODES

In light of its superior performance on our crafted benchmark SketchEval, we are keen to delve into the practicality of CODES in boosting programming efficiency. We thus develop “CODES Extended”, a VSCode extension based on CODES. To evaluate its practicality, we invite 30 participants to use CODES Extended and other competitive code assistants to implement two real-world engineering projects. We monitor their entire implementation process to analyze the advantages and disadvantages of different code assistants. This section first introduces all the code assistants participating in the evaluation, then describes the two projects that need to be implemented, followed by the procedure of our empirical study, and finally presents the evaluation results.

8.1 Code Assistants

- **Python Extended**¹⁵: This VSCode extension employs a rule-based strategy to enhance Python programming by providing API-level code suggestions.
- **GitHub Copilot**¹⁶: It is a VSCode extension driven by LLMs, co-developed by GitHub and OpenAI. It can improve coding efficiency by automatically generating line-level or function-level code snippets based on the context of the repository.
- **ChatDev Extended**: We deploy ChatDev [34] to implement this VSCode extension. It is capable of generating an entire code repository by allowing GPT-3.5 to play different roles (e.g., CEO, CTO, CFO, and programmer). Based on the generated repository, we allow users to edit its content to achieve their final goal.
- **CODES Extended**: CODES includes two versions: prompt engineering (PE) and supervised fine-tuning (SFT). So, we develop VSCode extensions for these two versions separately based on CodeLlama 34B. The extensions can generate an entire code repository through a three-layer sketch. We allow the user to edit the generated content of each layer before proceeding to the next layer. Users can realize their requirements through interactions with VSCode extensions or manual modifications.

8.2 Two Real-World Engineering Projects

To evaluate the practicality of the above code assistants, we design two projects for participants to implement. We provide brief descriptions of the projects below. For additional details, please visit the provided links.

- **Gomoku (easy)**¹⁷: This project is a two-player board game played on a 15x15 grid. The objective of the game is to be the first player to get five of their pieces in a row, either horizontally, vertically, or diagonally. This game needs to be implemented using the tkinter library¹⁸.
- **Blog (hard)**¹⁹: This project is a blog system. The features of this blog mainly include registration, login, blog management, and comment management. This blog requires to be implemented using the Flask framework²⁰.

8.3 Procedure of Our Empirical Study

8.3.1 Pipeline of User Study. For each of the five code assistants, we invite six participants to implement the two projects, three per project. In total, we invite thirty participants. For fairness, we ensure that these participants have similar backgrounds: both master students from the same university, both software engineering-related

¹⁵<https://github.com/tushortz/vscode-Python-Extended>

¹⁶<https://github.com/features/copilot>

¹⁷<https://github.com/NL2Code/CodeS/tree/main/projects/Gomoku>

¹⁸<https://docs.python.org/3/library/tkinter.html>

¹⁹<https://github.com/NL2Code/CodeS/tree/main/projects/Blog>

²⁰<https://flask.palletsprojects.com>

majors, and both with over two years of Python experience. We also invite three Ph.D. students as proctors to ensure the process progressed orderly. We systematically track and monitor their entire implementation process.

- **Firstly**, we record the average completion time across three participants. We also separately record the time they spent reading and writing code²¹.
- **Secondly**, we count the total number of requesting code assistants, and calculate the percentage of code tokens automatically generated by assistants.
- **Thirdly**, we count their common error types and invite all participants to evaluate the code assistants used.
- **Fourthly**, after all participants have completed their projects, we analyze the output code repositories in detail. Specifically, we count the number of directories, files, lines, classes, and functions for the generated code repositories.
- **Fifthly**, we use Pylint²² to measure the code quality of the generated repositories (scoring from 0 to 10) and invite three proctors to manually review them (scoring from 0 to 100).

8.3.2 Details of User Study.

- **User guideline briefing:** We dedicate a 100 minutes pre-training-session, including informing participants about the experimental environment, constraints, and objectives, as well as familiarizing them with the use of code assistants. Participants are requested to complete the project independently, but allowed access to the internet.
- **User Behavior Monitoring:** For all code assistants, we track user behavior via VSCode logs. Our developed plugin records real-time operations (e.g., model queries) and repository status (e.g., error reports) with timestamps.
- **Repository Completion Criteria:** Participants are requested to submit a completed project without runtime and functional errors, for which three proctors check the completion status and grade scores.
- **Support and Resources:** For the Blog project, we provide participants with detailed database SQL files and migrations²³, and clarify all database fields for them.
- **Other Details:** In the design of CODES Extended, we generate non-Python content by harnessing CODES's intrinsic capabilities via prompt engineering. We invite a total of thirty participants, none of whom are authors.

8.4 Results

Table 8 shows the performance of five code assistants on two real-world projects. It can be observed that these LLM-based assistants significantly outperform the rule-based one. For instance, Python Extended takes 4.2 hours to complete Gomoku, whereas GitHub Copilot and CODES Extended (SFT) complete the task in only 1.6 and 1.9 hours, respectively. This observation underscores the practicability of LLM-based assistants in accomplishing real-world NL2Repo tasks. Additionally, while ChatDev Extended achieves decent performance on Gomoku, it fails on the more intricate project Blog. In contrast, CODES Extended demonstrates a more stable performance on both Gomoku and Blog projects, highlighting its effectiveness and reliability in addressing NL2Repo tasks with varied difficulties. Compared with Copilot, users often spend far more time reading code than writing in the process of using CODES Extended. Specifically, Copilot takes 0.9h/0.7h to read/write code, versus 1.5h/0.4h for CODES Extended (SFT). After re-examining the implementation process, we discover that CODES Extended often generates more detailed code at once, which increases the workload of reading. However, once the repository

²¹More than ten seconds without any changes to the code is seen as reading, and vice versa for writing.

²²<https://pypi.org/project/pylint>

²³<https://flask-migrate.readthedocs.io>

Table 8. Results for five code assistants in two projects: **R./W.** for read/write time, **Req.** for the number of requesting assistants, **Code Prop.** for the auto-generated code token proportion, and **n/a** for projects not completed within 2.5 days.

Code Assistants		Python Extended	ChatDev Extended	GitHub Copilot	CodeS Extended (PE)	CodeS Extended (SFT)
Type		Rule	LLM	LLM	LLM	LLM
Base Model		-	GPT-3.5	-	CodeLlama 34B	CodeLlama 34B
Gomoku (Easy)	Time	4.2h	2.1h	1.6h	2.2h	1.9h
	R./W.	1.4h/2.8h	1.5h/0.6h	0.9h/0.7h	1.7h/0.5h	1.5h/0.4h
	Req.	-	103	66	73	60
	Code Prop.	-	67%	52%	63%	67%
Blog (Hard)	Time	2.3d	n/a	1.5d	1.9d	1.8d
	R./W.	0.7d/1.6d	n/a	0.8d/0.7d	1.3d/0.6d	1.3d/0.5d
	Req.	-	n/a	316	263	247
	Code Prop.	-	n/a	45%	50%	52%

Table 9. Statistics of the repositories generated by five code assistants. **#D./F./L.** means the number of directories, files, and lines; **#c./f.** denotes the number of class and function; **Q.** represents the code quality score from Pylint; **S.** indicates grade scores from three proctors.

Tool	Gomoku (Easy)				Blog (Hard)			
	#D./F./L.	#c./f.	Q.	S.	#D./F./L.	#c./f.	Q.	S.
Reference Repository	0/2/101	3/12	8.4	-	21/29/1,859	28/119	8.3	-
Python Extended	0/1/94	2/9	5.7	74	13/20/1,562	21/112	6.1	72
ChatDev Extended	0/3/125	3/14	6.3	83	n/a	n/a	n/a	n/a
GitHub Copilot	0/1/104	2/12	6.8	77	13/21/1,615	24/122	6.5	75
CodeS Extended (PE)	0/3/146	3/15	6.5	77	18/27/1,834	25/117	6.4	73
CodeS Extended (SFT)	0/2/133	4/17	7.2	84	19/26/1,774	27/120	6.9	77

generated by CODES Extended is understood, users will be able to accomplish their requirements in a shorter time. Regarding Copilot, its meticulous product design significantly enhances user experience [9, 30], contributing to the reduced time required to complete the two projects, which is also empirically evidenced in Fig. 5. This suggests that our plugin still has room for improvement in product design and other aspects.

To evaluate the degree to which the code assistants adhere to human coding style, in Table 9, we compare the statistics between the code assistants' generated repositories and the reference repository. It can be observed that CODES Extended (SFT) consistently receives the highest score on both the Gomoku and Blog projects. Compared with other code assistants, the repository structures generated by CODES Extended (PE) and (SFT) are more aligned with the reference repository, particularly for the hard project Blog. These observations further highlight the practicality of CODES in assisting real-world software development tasks.

To further compare the capability of CODES Extended and Copilot, we count the occurrences of various error types for each assistant in Fig. 6. Our analysis reveals that CODES Extended significantly outperforms Copilot in handling dependency-related errors: ImportError, FileNotFoundError, and ModuleNotFoundError, which highlights the advantages of CODES Extended in handling repository-level cross-file, and cross-module

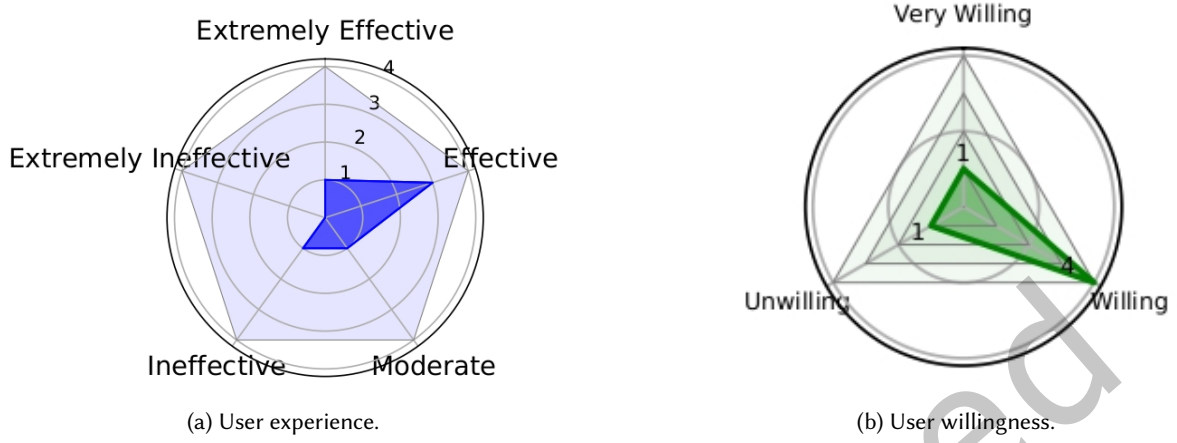


Fig. 5. Survey results from 6 participants on CODES Extended (a) experience and (b) willingness.

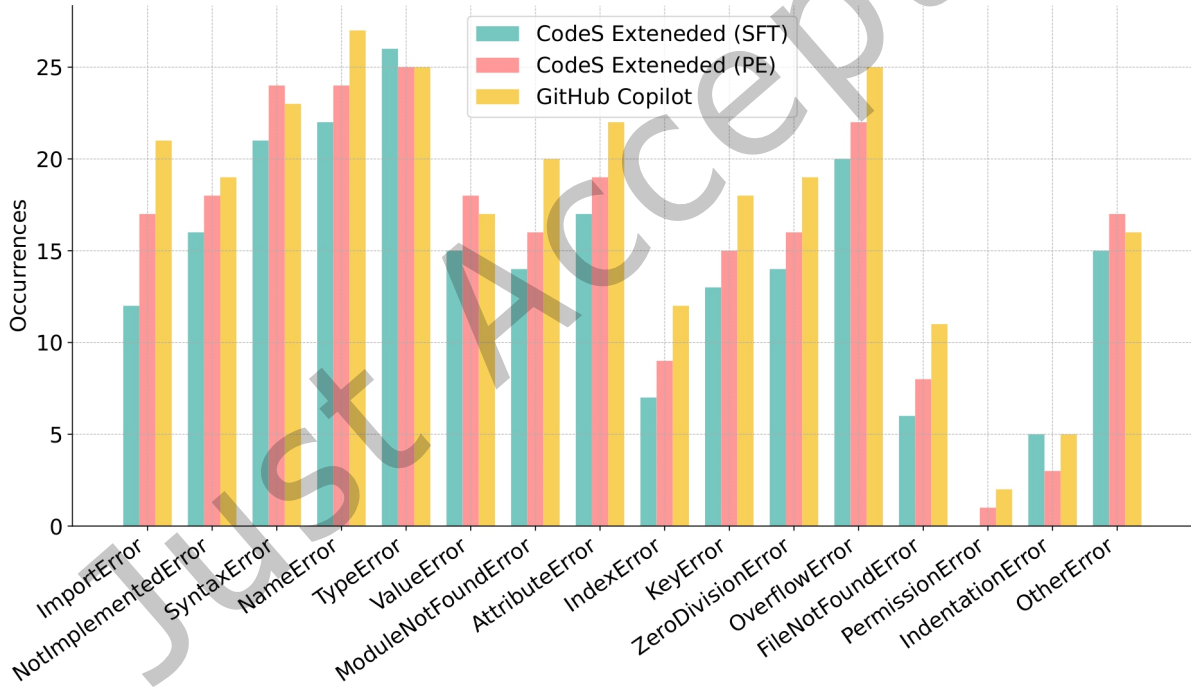


Fig. 6. The number of occurrences of various error types under different code assistants.

function invocations. Additionally, CODES Extended exhibits fewer occurrences of most other errors compared to Copilot, such as NameError, AttributeError, and OverflowError. The superior performance can be attributed to CODES Extended’s multi-layer decomposition and comprehensive understanding of repository structures.

In summary, our analysis highlights the advantages of CODES Extended in maintaining robust and error-free repositories, making it a valuable tool for developers aiming for higher code quality and stability.

After completing the project, we conduct a user survey with six participants who used the CODES Extended. The survey results in Fig. 5 (a) show that most participants deem the plugin we developed to be effective, further proving the effectiveness and practicality of CODES. Meanwhile, we also note a participant who deem CODES Extended ineffective. Further inquiry with this participant reveal that CODES Extended falls short in product design compared to the commercial product GitHub Copilot. This conclusion is also evident from the results above, highlighting substantial potential for improving the design of our developed VSCode plugin.

In addition to user experience, we also surveyed participants' willingness to continue using CODES Extended in future projects, as shown in Fig. 5 (b). The results indicate that the majority of users expressed a clear willingness to adopt CODES Extended for subsequent development tasks, with four out of six participants selecting "Willing" and one selecting "Very Willing." Only one participant reported an unwillingness to continue using the tool. This positive feedback demonstrates not only the practical value of CODES Extended in real-world scenarios, but also its potential for broader adoption among developers. The high willingness to reuse the tool further validates the effectiveness and user satisfaction of our approach, while also highlighting areas for future improvement to address the concerns of less satisfied users.

9 RELATED WORK

The task of NL2Repo necessitates long inputs as well as long structured outputs, which presents new and significant challenges:

NL2Repo requires handling long contexts. Most of the existing language models support context window sizes ranging from 4k to 32k [6, 8, 13, 22, 23, 31, 41, 47, 54]. Some support longer context up to 128K (Gemini), 200K (Claude 2.1) [3, 14, 27, 37]. Rotary Position Embedding (ROPE) [40] or the technique that concatenates multiple windows [44, 53] can extend the context window size effectively. However, even with the largest context window size, it is not sufficient for repository-level tasks due to the large size of code bases, documentation, and discussions.

NL2Repo requires efficient instruction fine-tuning. Instruction fine-tuning has been proved to be effective for multiple downstream code tasks [28, 38]. It is also used to enhance a wide variety of base models, such as WizardCoder [28], CodeLlama-Instruct [37], and DeepSeek-Coder-Instruct [14]. For complex tasks with long inputs and outputs. It not only has a longer fine-tuning time for one data instance but also needs more tuning data. It is usually not affordable to do a full parameter tuning. To reduce cost, LongLora [7] proposes sparse local attention which can finish fine-tuning for 32K window in around 11 hours with 8 A100 GPUs.

Benchmarks for complex coding tasks. Benchmarks for code generation are designed with different levels of context information but also target different granularity of code snippets. CoNaLa [45] targets line completion. HumanEval [6], APPs [16], CONCODE [18], and CoderEval [46] focus function generation. Moreover, RepoEval [52], CrossCodeEval [10], and RepoBench [25] focus on cross-file line/function-level code generation. Beyond lines and functions, class-level [11], and patch-level [19] generation is drawing more and more attention. This paper is the first to propose a repository-level benchmark SketchEval for evaluating the NL2Repo tasks.

Towards complex coding tasks with LLM-centered systems. Tasks like NL2Repo may be too hard for a single LLM to solve. Iterative generation with feedback from tools or humans seems to be mandatory. It may also require the capability of reasoning and the decomposition of tasks. Like Parsel [50], it reasons about code in the form of call graphs. Besides reasoning, feedback from test will be critical for correctness. Executing tests can identify the wrong generation in the early stage [36, 50]. Still, iterating only with tools or a "single-minded" LLM is not sufficient. New perspectives are also important for problem solving, either provided by different LLMs or humans, like ChatDev [34] utilizes communications between different agents, ANPL [17] interacts with humans.

The most recent work Devin [1] shows the potential of fully automated software development. This paper uses the sketch idea to empower LLMs to address the complex NL2Repo task, achieving decent performance.

10 THREATS TO VALIDITY

This section will analyze the threats to the validity of our approach. Specifically, we discuss both the internal and external threats along with their mitigating factors.

10.1 Internal Threats

- The CODES comprises three modules (i.e., RepoSketcher, FileSketcher, and SketchFiller), which respectively correspond to the three layers of the code repository architecture. However, the structural hierarchy within the repository is more intricate than these three layers, which has not been fully taken into consideration. We would like to explore how to implement NL2Repo via a more fine-grained sketch.
- The invocation relationships and dependencies in real-world engineering repositories are typically complex. While CODES has mechanisms to handle cross-file dependencies, our empirical results (Fig. 6) show that dependency-related errors (e.g., ImportError and NameError) still occur frequently. These challenges arise from several key factors. First, circular dependencies between modules require careful ordering of file generation. Second, dynamic imports and conditional imports are context-dependent and difficult to resolve statically. Third, complex package version requirements may conflict across different modules. To address these limitations, we are exploring several potential improvements: incorporating static analysis techniques like call graph analysis [15] to better understand dependency relationships; implementing a two-pass generation strategy where the first pass focuses on interface consistency and the second pass handles implementation details; and developing specialized modules for dependency resolution that can handle complex package management scenarios.
- Although we have decomposed the NL2Repo task with the sketch and enabled ROPE [40] during fine-tuning, the limited long-context capability of LLMs remains a constraint for overly large repositories. A possible solution is to highly condense information using LLMs' summarization capabilities, which remains to be explored in the future.
- Our approach, which is based on the multi-layer sketch, may be susceptible to cascading errors. Fortunately, human intervention can mitigate this issue by supervising the generated results of each layer. The experimental results presented in Section 8 can also prove the practical potential of CODES.
- In this work, we use README.md as the primary input format for natural language requirements. While README.md provides a convenient starting point for exploring NL2Repo tasks, it may not always be the most optimal or comprehensive way to express software requirements. Future work could explore alternative input formats such as structured requirement specifications, user stories, or interactive requirement gathering sessions. Additionally, we could investigate how to better process and extract relevant information from README.md files, as they often contain content unrelated to actual requirements.
- The current approach assumes that detailed implementation documentation (e.g., installation instructions, command-line flags, and usage examples) is available in the README.md before code generation. While this assumption helps ensure the generated code is installable and runnable, it may not always reflect real-world scenarios where such documentation is typically written after the code is implemented. This limitation could affect the practical applicability of CODES in scenarios where comprehensive documentation is not available upfront. Future work could explore ways to handle cases where implementation details need to be determined during the development process.
- While we have proposed SketchBLEU for evaluating NL2Repo effectiveness, automatically assessing the quality of generated repositories remains a significant challenge. We envision several promising directions

for enhancing evaluation methods. First, feature coverage analysis could be developed to map requirements in README files to actual code implementations, possibly using techniques from requirement traceability research. Second, semantic correctness verification could employ program analysis tools to verify not just syntactic correctness but also semantic properties like data flow consistency and API usage patterns. Third, test case generation could automatically create tests from README examples and usage scenarios to verify functional correctness. Fourth, repository structure assessment could develop metrics to evaluate the quality of the overall repository architecture, including modularity, coupling, and cohesion. Each of these directions presents its own challenges, such as the ambiguity in natural language requirements, the difficulty in automatically generating meaningful test cases, and the subjective nature of architectural quality. Future work could explore these directions while carefully considering their limitations and trade-offs.

10.2 External Threats

- Despite our best efforts to gather the most current open-source repositories (2023.08.01~) for evaluation, some recently released models, such as StarCoder2 (~2023.09.06), still possibly have seen them during pre-training, due to the rapid evolution of LLMs.
- Due to the task's multifaceted nature, it is challenging to devise a metric that is completely fair and universally accepted. However, other than benchmark-based experiments evaluated with SketchEval, we also conduct feedback-based experiments to provide a more diverse and comprehensive evaluation.
- Due to limited resources, we conducted a user study with only 30 participants, with 5 participants assigned to each assistant. To achieve a more comprehensive evaluation, we plan to publish CODES in VSCode#Marketplace.
- This paper focuses on Python-version general code repositories. In future work, we will adopt the idea of CODES to more languages (e.g., Java and C++) and more vertical programming tasks (e.g., web development and robot control). Interestingly, CODES's Python version can be generalized to other languages, with details in Section 7.3.2.
- For the proposed evaluation metric SketchBLEU, we acknowledge that there is a potential threat regarding its correlation with human judgment. To mitigate this concern, we conducted a preliminary study where we asked 3 experienced developers to manually evaluate 5 repositories from SketchEval using a 5-point Likert scale. The results showed a moderate positive correlation (Pearson's $r = 0.68$) between SketchBLEU scores and human ratings. While this provides some validation, we recognize that a more comprehensive study with a larger sample size would be needed to fully establish the metric's reliability. We plan to conduct such a study in future work and potentially refine SketchBLEU based on the findings.

11 CONCLUSION AND FUTURE WORK

This paper proposes a new software engineering task, namely natural language to code repository (NL2Repo). To address this task, we propose a framework called CODES. It can decompose a complex NL2Repo task into multiple sub-tasks via a multi-layer sketch, and then solve them layer by layer. To evaluate CODES on the NL2Repo task, we craft a new benchmark SketchEval, and provide an evaluation metric SketchBLEU. Furthermore, we develop a VSCode plugin for CODES and invite 19 participants to conduct a comprehensive empirical study. Extensive experiments have proved the effectiveness and practicality of CODES.

As the first explorers, we use LLMs to build a basic framework (CODES) for NL2Repo, leveraging LLMs' powerful natural-language capabilities and extensive knowledge. However, CODES also allows for future extensions to integrate with third-party tools (e.g., rule-based approaches) via the multi-layer sketch. We plan to integrate

rule-based and human-in-the-loop strategies for vertical domains (e.g., VUE²⁴ and SpringBoot²⁵), solving more focused tasks.

ACKNOWLEDGMENTS

This research was supported by the National Key Research and Development Program of China, under Grant No. 2022ZD0120201 - “Unified Representation and Knowledge Graph Construction for Science Popularization Resources”.

REFERENCES

- [1] [n. d.]. Devin: the first AI software engineer. <https://www.cognition.ai/blog/introducing-devin> ([n. d.]).
- [2] 2023. AgentGPT. <https://github.com/reworkd/AgentGPT> (2023).
- [3] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774* (2023).
- [4] AI Anthropic. 2024. The claude 3 model family: Opus, sonnet, haiku. *Claude-3 Model Card 1* (2024).
- [5] Neelakantan Arvind, V Le Quoc, Abadi Martin, McCallum Andrew, Amodei Dario, et al. 2017. Learning a natural language interface with neural programmer. In *5th International Conference on Learning Representations, ICLR 2017, Toulon, France, April 24–26, 2017, Conference Track Proceedings*.
- [6] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374* (2021).
- [7] Yukang Chen, Shengju Qian, Haotian Tang, Xin Lai, Zhijian Liu, Song Han, and Jiaya Jia. [n. d.]. LongLoRA: Efficient Fine-tuning of Long-Context Large Language Models. In *The Twelfth International Conference on Learning Representations*.
- [8] Fenia Christopoulou, Gerasimos Lampouras, Milan Gritta, Guchun Zhang, Yinpeng Guo, Zhongqi Li, Qi Zhang, Meng Xiao, Bo Shen, Lin Li, et al. 2022. Pangu-coder: Program synthesis with function-level language modeling. *arXiv preprint arXiv:2207.11280* (2022).
- [9] Arghavan Moradi Dakhel, Vahid Majdinasab, Amin Nikanjam, Foutse Khomh, Michel C Desmarais, and Zhen Ming Jack Jiang. 2023. Github copilot ai pair programmer: Asset or liability? *Journal of Systems and Software* 203 (2023), 111734.
- [10] Yangruibo Ding, Zijian Wang, Wasi Ahmad, Hantian Ding, Ming Tan, Nihal Jain, Murali Krishna Ramanathan, Ramesh Nallapati, Parminder Bhatia, Dan Roth, et al. 2024. Crosscodeeval: A diverse and multilingual benchmark for cross-file code completion. *Advances in Neural Information Processing Systems* 36 (2024).
- [11] Xueying Du, Mingwei Liu, Kaixin Wang, Hanlin Wang, Junwei Liu, Yixuan Chen, Jiayi Feng, Chaofeng Sha, Xin Peng, and Yiling Lou. 2024. Evaluating large language models in class-level code generation. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. 1–13.
- [12] Kevin Ellis, Catherine Wong, Maxwell Nye, Mathias Sablé-Meyer, Lucas Morales, Luke Hewitt, Luc Cary, Armando Solar-Lezama, and Joshua B Tenenbaum. 2021. Dreamcoder: Bootstrapping inductive program synthesis with wake-sleep library learning. In *Proceedings of the 42nd acm sigplan international conference on programming language design and implementation*. 835–850.
- [13] Daniel Fried, Armen Aghajanyan, Jessy Lin, Sida Wang, Eric Wallace, Freda Shi, Ruiqi Zhong, Scott Yih, Luke Zettlemoyer, and Mike Lewis. [n. d.]. InCoder: A Generative Model for Code Infilling and Synthesis. In *The Eleventh International Conference on Learning Representations*.
- [14] Daya Guo, Qihao Zhu, Dejian Yang, Zhenda Xie, Kai Dong, Wentao Zhang, Guanting Chen, Xiao Bi, Yu Wu, YK Li, et al. 2024. DeepSeek-Coder: When the Large Language Model Meets Programming—The Rise of Code Intelligence. *arXiv preprint arXiv:2401.14196* (2024).
- [15] Mary W Hall and Ken Kennedy. 1992. Efficient call graph analysis. *ACM Letters on Programming Languages and Systems (LOPLAS)* 1, 3 (1992), 227–242.
- [16] Dan Hendrycks, Steven Basart, Saurav Kadavath, Mantas Mazeika, Akul Arora, Ethan Guo, Collin Burns, Samir Puranik, Horace He, Dawn Song, et al. [n. d.]. Measuring Coding Challenge Competence With APPS. In *Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track (Round 2)*.
- [17] Di Huang, Ziyuan Nan, Xing Hu, Pengwei Jin, Shaohui Peng, Yuanbo Wen, Rui Zhang, Zidong Du, Qi Guo, Yewen Pu, et al. 2024. ANPL: towards natural programming with interactive decomposition. *Advances in Neural Information Processing Systems* 36 (2024).
- [18] Srinivasan Iyer, Ioannis Konstas, Alvin Cheung, and Luke Zettlemoyer. 2018. Mapping Language to Code in Programmatic Context. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*. 1643–1652.

²⁴<https://vuejs.org>

²⁵<https://spring.io/projects/spring-boot>

- [19] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R Narasimhan. [n. d.]. SWE-bench: Can Language Models Resolve Real-world Github Issues?. In *The Twelfth International Conference on Learning Representations*.
- [20] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles*. 611–626.
- [21] Richard Levinson. 1995. A general programming language for unified planning and control. *Artificial Intelligence* 76, 1-2 (1995), 319–375.
- [22] Raymond Li, Yangtian Zi, Niklas Muennighoff, Denis Kocetkov, Chenghao Mou, Marc Marone, Christopher Akiki, Li Jia, Jenny Chim, Qian Liu, et al. [n. d.]. StarCoder: may the source be with you! *Transactions on Machine Learning Research* ([n. d.]).
- [23] Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, et al. 2022. Competition-level code generation with alphacode. *Science* 378, 6624 (2022), 1092–1097.
- [24] Tianyu Liu, Yuchen Eleanor Jiang, Nicholas Monath, Ryan Cotterell, and Mrinmaya Sachan. 2022. Autoregressive Structured Prediction with Language Models. In *Findings of the Association for Computational Linguistics: EMNLP 2022*. 993–1005.
- [25] Tianyang Liu, Canwen Xu, and Julian McAuley. [n. d.]. RepoBench: Benchmarking Repository-Level Code Auto-Completion Systems. In *The Twelfth International Conference on Learning Representations*.
- [26] Ilya Loshchilov and Frank Hutter. [n. d.]. Decoupled Weight Decay Regularization. In *International Conference on Learning Representations*.
- [27] Anton Lozhkov, Raymond Li, Loubna Ben Allal, Federico Cassano, Joel Lamy-Poirier, Nouamane Tazi, Ao Tang, Dmytro Pykhtar, Jiawei Liu, Yuxiang Wei, et al. 2024. Starcoder 2 and the stack v2: The next generation. *arXiv preprint arXiv:2402.19173* (2024).
- [28] Ziyang Luo, Can Xu, Pu Zhao, Qingfeng Sun, Xiubo Geng, Wenxiang Hu, Chongyang Tao, Jing Ma, Qingwei Lin, and Daxin Jiang. [n. d.]. WizardCoder: Empowering Code Large Language Models with Evol-Instruct. In *The Twelfth International Conference on Learning Representations*.
- [29] Lili Mou, Ge Li, Lu Zhang, Tao Wang, and Zhi Jin. 2016. Convolutional neural networks over tree structures for programming language processing. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 30.
- [30] Nhan Nguyen and Sarah Nadi. 2022. An empirical evaluation of GitHub copilot’s code suggestions. In *Proceedings of the 19th International Conference on Mining Software Repositories*. 1–5.
- [31] Erik Nijkamp, Bo Pang, Hiroaki Hayashi, Lifu Tu, Huan Wang, Yingbo Zhou, Silvio Savarese, and Caiming Xiong. [n. d.]. CodeGen: An Open Large Language Model for Code with Multi-Turn Program Synthesis. In *The Eleventh International Conference on Learning Representations*.
- [32] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. 2022. Training language models to follow instructions with human feedback. *Advances in neural information processing systems* 35 (2022), 27730–27744.
- [33] S Porter. 2003. Sketching, concept development and automotive design. *Design studies* 24, 2 (2003), 135–153.
- [34] Chen Qian, Xin Cong, Cheng Yang, Weize Chen, Yusheng Su, Juyuan Xu, Zhiyuan Liu, and Maosong Sun. 2023. Communicative agents for software development. *arXiv preprint arXiv:2307.07924* (2023).
- [35] Shuo Ren, Daya Guo, Shuai Lu, Long Zhou, Shujie Liu, Duyu Tang, Neel Sundaresan, Ming Zhou, Ambrosio Blanco, and Shuai Ma. 2020. Codebleu: a method for automatic evaluation of code synthesis. *arXiv preprint arXiv:2009.10297* (2020).
- [36] Tal Ridnik, Dedy Kredo, and Itamar Friedman. 2024. Code generation with alphacodium: From prompt engineering to flow engineering. *arXiv preprint arXiv:2401.08500* (2024).
- [37] Baptiste Roziere, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Tal Remez, Jérémy Rapin, et al. 2023. Code llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950* (2023).
- [38] Bo Shen, Jiaxin Zhang, Taihong Chen, Daoguang Zan, Bing Geng, An Fu, Muhan Zeng, Ailun Yu, Jichuan Ji, Jingyang Zhao, et al. 2023. Pangu-coder2: Boosting large language models for code with ranking feedback. *arXiv preprint arXiv:2307.14936* (2023).
- [39] Armando Solar-Lezama. 2008. *Program synthesis by sketching*. University of California, Berkeley.
- [40] Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. 2024. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing* 568 (2024), 127063.
- [41] Yue Wang, Weishi Wang, Shafiq Joty, and Steven CH Hoi. 2021. CodeT5: Identifier-aware Unified Pre-trained Encoder-Decoder Models for Code Understanding and Generation. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*. 8696–8708.
- [42] Charles Wells. 1990. A generalization of the concept of sketch. *Theoretical Computer Science* 70, 1 (1990), 159–178.
- [43] Hui Yang, Sifu Yue, and Yunzhong He. 2023. Auto-gpt for online decision making: Benchmarks and additional opinions. *arXiv preprint arXiv:2306.02224* (2023).
- [44] Howard Yen, Tianyu Gao, and Danqi Chen. 2024. Long-context language modeling with parallel context encoding. *arXiv preprint arXiv:2402.16617* (2024).
- [45] Pengcheng Yin, Bowen Deng, Edgar Chen, Bogdan Vasilescu, and Graham Neubig. 2018. Learning to mine aligned code and natural language pairs from stack overflow. In *Proceedings of the 15th international conference on mining software repositories*. 476–486.

- [46] Hao Yu, Bo Shen, Dezhi Ran, Jiaxin Zhang, Qi Zhang, Yuchi Ma, Guangtai Liang, Ying Li, Qianxiang Wang, and Tao Xie. 2024. Codereval: A benchmark of pragmatic code generation with generative pre-trained models. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*. 1–12.
- [47] Daoguang Zan, Bei Chen, Dejian Yang, Zeqi Lin, Minsu Kim, Bei Guan, Yongji Wang, Weizhu Chen, and Jian-Guang Lou. 2022. CERT: Continual Pre-training on Sketches for Library-oriented Code Generation. In *Proceedings of the Thirty-First International Joint Conference on Artificial Intelligence, IJCAI-22*, Lud De Raedt (Ed.). International Joint Conferences on Artificial Intelligence Organization, 2369–2375. <https://doi.org/10.24963/ijcai.2022/329> Main Track.
- [48] Daoguang Zan, Bei Chen, Fengji Zhang, Dianjie Lu, Bingchao Wu, Bei Guan, Wang Yongji, and Jian-Guang Lou. 2023. Large Language Models Meet NL2Code: A Survey. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 7443–7464.
- [49] Daoguang Zan, Ailun Yu, Bo Shen Wei Liu, Shaoxin Lin, Yongshun Gong, Yafen Yao, Yan Liu, Bei Guan, Weihua Luo, Yongji Wang, Qianxiang Wang, and Lizhen Cui. 2024. CodeM: Less Data Yields More Versatility via Ability Matrix. In *The 62nd Annual Meeting of the Association for Computational Linguistics*. <https://openreview.net/forum?id=FhpWlls7V3>
- [50] Eric Zelikman, Qian Huang, Gabriel Poesia, Noah Goodman, and Nick Haber. 2023. Parsel: Algorithmic Reasoning with Language Models by Composing Decompositions. *Advances in Neural Information Processing Systems* 36 (2023), 31466–31523.
- [51] Biao Zhang, Zhongtao Liu, Colin Cherry, and Orhan Firat. [n. d.]. When Scaling Meets LLM Finetuning: The Effect of Data, Model and Finetuning Method. In *The Twelfth International Conference on Learning Representations*.
- [52] Fengji Zhang, Bei Chen, Yue Zhang, Jacky Keung, Jin Liu, Daoguang Zan, Yi Mao, Jian-Guang Lou, and Weizhu Chen. 2023. RepoCoder: Repository-Level Code Completion Through Iterative Retrieval and Generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. 2471–2484.
- [53] Peitian Zhang, Zheng Liu, Shitao Xiao, Ninglu Shao, Qiwei Ye, and Zhicheng Dou. 2024. Soaring from 4K to 400K: Extending LLM’s Context with Activation Beacon. *arXiv preprint arXiv:2401.03462* (2024).
- [54] Qinkai Zheng, Xiao Xia, Xu Zou, Yuxiao Dong, Shan Wang, Yufei Xue, Lei Shen, Zihan Wang, Andi Wang, Yang Li, et al. 2023. Codegeex: A pre-trained model for code generation with multilingual benchmarking on humaneval-x. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 5673–5684.
- [55] Yaowei Zheng, Richong Zhang, Junhao Zhang, Yanhan Ye, and Zheyang Luo. 2024. Llamafactory: Unified efficient fine-tuning of 100+ language models. *arXiv preprint arXiv:2403.13372* (2024).

Received 20 July 2024; revised 25 August 2025; accepted 14 September 2025